

QUBO for deterministic-8 and flow accumulation: another edge between Quantum Computation and Hydrology*

Jaime Anguiano Olarra[†]
(Dated: May 7, 2026)

Flood risk management has become a major area of research for those institutions and administrations endorsed with forecasting and damage mitigation, not only due to precipitation but also due to infrastructure failure. *Deterministic-8* is a standard algorithm in the sequence of routines applied to predict the way a volume of fluid will distribute over the land accessible to it. The continuous version of the problem is a dynamical one with constrictions for which a traditional Lagrange multipliers approach is natural. A lattice version of it is NP-complete complexity-wise and computationally is well suited for a quantum approach the latter being the object of this paper.

I. INTRODUCTION:

Flood risk assessment and subsequent damage prevention is gaining attention in the recent years. Even more acute is the urge for enhancing the prediction tools available for the agents watching regions where the actual models are lacking or outdated due to new trends in precipitation patterns, land-use, obsolete infrastructures or new legislation (many plans are requiring longer *return periods*) in many circumstances, all these issues concurring all at once.

Dramatic flash-flood events like the one that hit Spain on October 29th, 2024, with over 230 deaths, and which could have been increased by one or two orders of magnitude had a nearby dam fail, as it seemed plausible at the time [1], are of special interest. *Deterministic-8* is a widely used routine in the pre-processing stages of hydrologic studies for flood planning and is provided in industry standard toolkits like SAGA[2]. The goal of the problem is forecasting how a certain amount of water initially localized over some area will spread in its surrounding area. Machine Learning Quantum Computing[3, 4] has just not entered yet fully in this active field of Hydrology but the problems posed are very suitable for its application. The rest of this manuscript is devoted to build a general Quadratic Unconstrained Binary Optimization (QUBO) representation on paper containing the Deterministic-8 method, applying it to selected cases to showcase its use, ending with conclusions.

II. THEORETICAL FORMULATION

We follow notation introduced by O'Malley[5] whenever possible.

A. Deterministic-8 General QUBO

Deterministic-8 is a method self-described by its own name, being the first solution anyone can think of when faced with deciding, like in a boardgame, how to move in a square grid from one square to the next one. Simply put: explore all the next cells where you can move to check if doing so brings you closer to your goal, moving to the most favorable ones. Several at the same time, or many times, when time presses, shifting to only one just like as a *one hot* problem. This is precisely how it is done in the SAGA tools[2], an industry standard in the field of pre-processing Digital Elevation Models for later use in the modeling software later involved (like HEC-RAS[6], QGIS, iRIC or Iber). This routine can be considered a checkpoint before any more detailed hydrologic analysis takes place.

In our present case, identification with the 2-D model discussed by O'Malley [5] is direct. Keeping his notation we just give a different interpretation to the magnitudes behind the variables. This is better seen through an example. We start at time $t = 0$ with a 3x3 grid of vertices which will be our potentially flooded area, and we set on the vertex $h_{2,2}^0$ our aquifer in [5] scheme. The variables are binary, $h_{i,j}^t$, has value 1 if it is filled and value 0 if it is empty, and indexing starts at 1: $i, j = 1, 2, 3, \dots$. Just like in knapsack, a multiplicative constant $\alpha_{i,j}^t$ accounts for the height of free surface of water of that cell. The area of each cell is unitary (a simplification which is also standard in industry as seen for example in how the accumulated flow is computed in SAGA). We will call this grid the *run-off layer*. Notice that up to this point the layer lacks edges. We tackle this problem immediately.

Define a second grid of the same dimensions which serves as a faithful representation of the terrain, the Digital Elevation Model in jargon. Position it right below the previous one giving a value of elevation $z_{i,j}^t$ to each vertex by the same binary variable times a multiplicative constant assignment but with the difference that in this case all vertices are non-zero. Setting the value for the edges equal to the difference between the values of the vertices they connect, subtracting the arrival vertex value (height) from the departure vertex value and removing all the vertices of this middle grid, we are left with a

* jaimeangola.github@gmail.com

[†] jaime.a.olarra@gmail.com

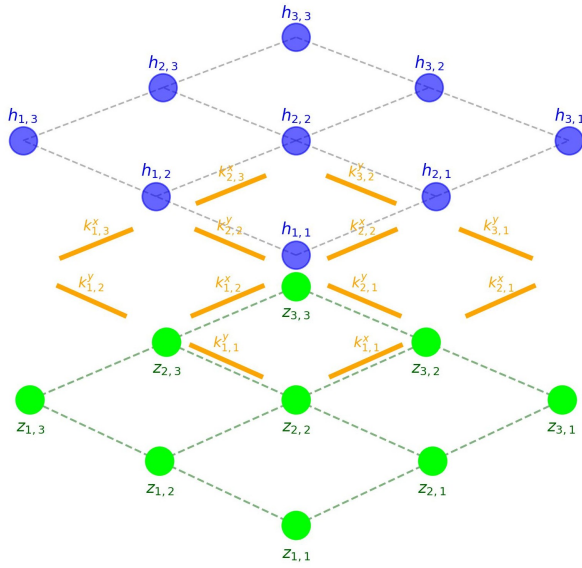


FIG. 1. The three 3x3 grid layers: The *run-off* layer in blue, on top. Its vertices are binary valued, the value of the heads are parameters $\alpha_{i,j}$ not shown here. The terrain layer in green at the bottom and a middle channel network obtained from signed differences in the aggregated values of head + height values of adjacent cells. Time subscripts have been omitted for simplicity.

subgraph which can be plugged into the *run-off* grid to supply the edges to the $h_{i,j}$ grid. These subgraph's values are signed, they represent the gravitational gradients. We will reuse O'Malley's label $k_{i,j}^{\mu}$, even though here it is not permeability. The subscript μ stands for the different spatial dimensions along the water moves in the plane, taking values from $\mu = x, y$ whereas the subscripts indicate which vertex was taken as departure, so f.ex. $k_{1,2}^x$ uniquely identifies the edge that moving along x , connects from the vertex $h_{1,2}^t$ to the vertex $h_{2,2}^t$. Thus we have constructed what in Hydrology is called a channel network.

The function to optimize is the negative of the potential energy. i.e. we want L to be as negative as we can:

$$L_{free} = -V = -\sum_{i,j} (\alpha_{i,j}^t + z_{i,j}^t) h_{i,j}^t \quad (1)$$

subject to two constraints. The first one is mass conservation:

$$\alpha_{i,j}^0 h_{i,j}^0 = \sum_{i,j} \alpha_{i,j}^t h_{i,j}^t \quad (2)$$

(water spreads, but no water is lost, this forces on us to set a lower bound to $\alpha_{i,j}$ where to stop covering more area which we set to 1). The second is that inertia alone drives the dynamics: water covers cells according to gravity, minimizing its overall gravitational energy, following general principles of Statistical Mechanics[7]. This potential to change vertex will be represented by an interaction term between the content of neighboring cells We

build the later using the following steps. The direction of movement is signed as we saw, for example for moving along the x axis:

$$\frac{z_{i+1,j}^t - z_{i,j}^t}{|z_{i+1,j}^t - z_{i,j}^t|} = \begin{cases} -1, & z_{i+1,j}^t < z_{i,j}^t \\ 0, & z_{i+1,j}^t = z_{i,j}^t \\ 1, & z_{i+1,j}^t > z_{i,j}^t \end{cases} \quad (3)$$

This factor serves as a switch to automatically turn *on* or *off* the inclusion of the interaction term. Only if it is *on* there is a gateway to lose energy by occupying new cells. Using the $k_{i,j}^{\mu}$ notation:

$$\frac{k_{i,j}^{\mu,t}}{|k_{i,j}^{\mu,t}|} = \hat{k}_{i,j}^{\mu,t} = \begin{cases} -1, & k_{i,j}^{\mu,t} < 0 \\ 0, & k_{i,j}^{\mu,t} = 0 \\ 1, & k_{i,j}^{\mu,t} > 0 \end{cases} \quad (4)$$

It is not obvious but this factor does not suffice for its purpose. The problem is that if an adjacent cell is at the same height it seems there is no reason for water to spill into it. But imagine that the previous cell has some water on it, it will spill till it reaches the lower bound for $\alpha_{i,j}^t$. This is realized by merely replacing the $z_{i,j}^t$ by $\alpha_{i,j}^t + z_{i,j}^t$, consider this done and accounted for by the $\hat{k}_{i,j}^{\mu,t}$. This leads to a bare Lagrangian interaction term that turns on when the right conditions apply allowing movement, which translates in activating new $h_{i,j}^t$ vertices flipping their value from their initial 0 to 1:

$$L_{int}^{bare} = -\frac{1}{2} \left(1 - \hat{k}_{i,j}^{\mu,t} \right) = \begin{cases} -1, & \text{if } \hat{k}_{i,j}^{\mu,t} = -1 \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

Notice that it embodies a binary variable in itself. Finally we need to encode by how much does it afflict the bare Lagrangean. In this case it is the change in overall potential energy:

$$L_{int} = -\frac{1}{2} \left(1 - \hat{k}_{i,j}^{\mu,t} \right) \sum_{i,j} [(\alpha_{i,j}^{t+1} + z_{i,j}^t) h_{i,j}^{t+1} - (\alpha_{i,j}^t + z_{i,j}^t) h_{i,j}^t] \quad (6)$$

And the full QUBO:

$$L = -\sum_{i,j} (\alpha_{i,j}^t + z_{i,j}^t) h_{i,j}^t - \frac{1}{2} \left(1 - \hat{k}_{i,j}^{\mu,t} \right) \sum_{i,j} [(\alpha_{i,j}^{t+1} + z_{i,j}^t) h_{i,j}^{t+1} - (\alpha_{i,j}^t + z_{i,j}^t) h_{i,j}^t] + \gamma \sum_{i,j} (\alpha_{i,j}^0 h_{i,j}^0 - \sum_{i,j} \alpha_{i,j}^t h_{i,j}^t) \quad (7)$$

which simplifies to:

$$L = -\frac{1}{2} \sum_{i,j} \left[\left((1-\gamma) + (1+\gamma) \hat{k}_{i,j}^{\mu,t} \right) \alpha_{i,j}^t + (1+\hat{k}_{i,j}^{\mu,t}) z_{i,j}^t \right] h_{i,j}^t - \left(1 - \hat{k}_{i,j}^{\mu,t} \right) \sum_{i,j} [(\alpha_{i,j}^{t+1} + z_{i,j}^t) h_{i,j}^{t+1} + \gamma \alpha_{i,j}^0 h_{i,j}^0] \quad (8)$$

B. Physical considerations

In this model no other physical effect is considered. Nonetheless see that it is legitimate to let the amount of water vary at anytime (*e.g.* rainfall, release of a dam) or rate by varying the $\alpha_{i,j}^t$ consistently. That the most efficient way to come to the ground-state is by flowing along the largest $\hat{k}_{i,j}$ s was not explicitly written in anyway and it is expected that this behavior will emerge naturally. Before moving onto the software implementation *per se*, it is worth mentioning that the diffusion of water discretized as in here is truly a good example of an Ising model where the interaction fits very well the next-neighbor condition as even if we were to address more complex fluid dynamics, the next-to-next neighbor structure is built-in for the problem of spreading of water flow, at least when far from supersonic or high cavitation regimes.

III. IN MACHINA IMPLEMENTATION DESCRIPTION

We have developed our model, now we apply it to some showcase examples growing in difficulty.

A. Showcase examples

1. 1-D Dam failure

We remove the j index as there is no need for it. This is the most simple model of dam failure. We have two units of water on the only cell (vertex) active and at some indeterminate time between time $t = 0$ and $t = 1$ we remove the constrain letting the water be able to flood the adjacent cell.

Initial state:

TERRAIN: $z_0^0 = z_1^0 = 1$ (our *Digital Elevation Model*)

$$[h_0^{t=0} = 1, \alpha_0^{t=0} = 2] \quad (9)$$

The only $\hat{k}_0^0 = \frac{(1+0)-(1+1)}{|(1+0)-(1+1)|} = -1$ so even if the channel network suggest that no flooding should be expected (the terrain is flat), the fact that water won't stand showing some kind of self-sustained "hydraulic jump" is effectively realized by the inclusion of the $\alpha_{i,j}^t$ as part of the switch function.

Final state:

$$[[h_0^{t=1} = 1, \alpha_0^{t=1} = 1], [h_1^{t=1} = 1, \alpha_1^{t=1} = 1]] \quad (10)$$

2. 3-D flooding

Initial state:

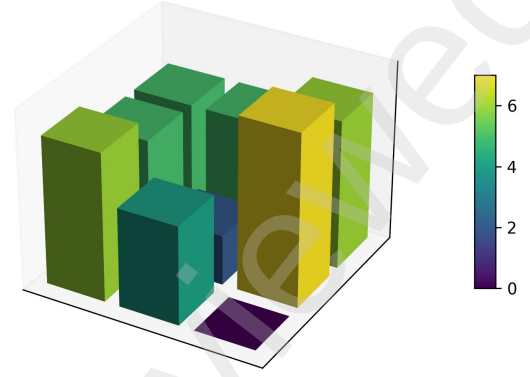


FIG. 2. Sample terrain

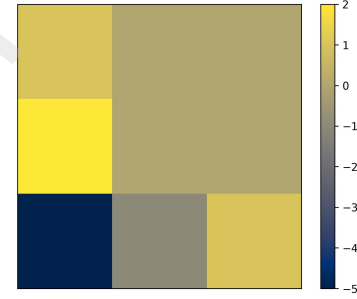


FIG. 3. Channel network

TERRAIN: This would be imported from Geopandas[8] for example.

$$z_{i,j}^0 = \begin{bmatrix} z_{1,3}^0 = 6 & z_{2,3}^0 = 5 & z_{3,3}^0 = 5 \\ z_{1,2}^0 = 7 & z_{2,2}^0 = 2 & z_{3,2}^0 = 5 \\ z_{1,1}^0 = 0 & z_{2,1}^0 = 4 & z_{3,1}^0 = 6 \end{bmatrix} \quad (11)$$

As we see, the aquifer is located at $h_{2,2}^0$ and filled to three units, i.e. $\alpha_{2,2}^0 = 3$, all the other $h_{i,j}^0$ are dry (have value 0 as well as all the other $\alpha_{i,j}^0$. The channel network obviously only gives us a path from $z_{2,2}^0$ to $z_{1,2}^0$ to $z_{1,1}^0$. If we include the height of the free surface we get:

$$\alpha_{i,j}^0 + z_{i,j}^0 = \begin{bmatrix} 6 & 5 & 5 \\ 7 & 5 & 5 \\ 0 & 4 & 6 \end{bmatrix} \quad (12)$$

We can dramatically enhance movement by breaking a dam changing the terrain, which we do setting $z_{1,2}^1 = 1$. So at time $t = 1$ a new state with lower energy could be:

State at $t = 1$:

$$[[h_{2,2}^{t=1} = 1, \alpha_{2,2}^{t=1} = 2], [h_{1,2}^{t=1} = 1, \alpha_{1,2}^{t=1} = 1]] \quad (13)$$

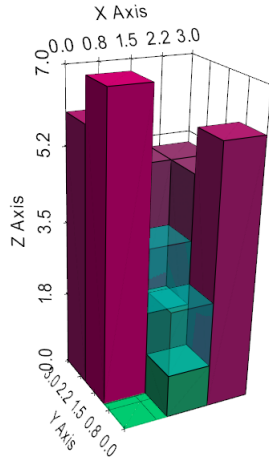


FIG. 4. Flow can be perceived in a less abstract way by means of this figure at an intermediate step.

And at time $t = \infty$ the final state needs to be the ground-state which actually can be achieved at $t=1$ if both cells z_{11}^1 and z_{12}^1 are flooded in one step:

Final state:

$$[[h_{1,1}^{t=\infty} = 1, \alpha_{1,1}^{t=\infty} = 2], [h_{1,2}^{t=\infty} = 1, \alpha_{1,2}^{t=\infty} = 1]] \quad (14)$$

This final time is the initial time of the next iteration of the algorithm in a larger grid. In fact, given a larger grid and controlling how much flow can move

at each timestep enriches the dynamics notably as what otherwise are accumulation points, the vertex $z_{1,1}^t$ in our grid, can be mere parts of a channel even with negligible Strahler order in an larger catchment area.

B. Conclusions

The simplicity of this proof of concept model makes it suitable to be coded using almost any framework for quantum computing at the present time. In D-Wave's Ocean SDK[9] this can be achieved using a quadratic model and an ExactSolver without requiring running it on actual quantum architectures. Considering the smallness of these showcases calls for sticking to the ExactSolver. A version of it using D-Wave's SimulatedAnnealingSampler() is available[10]. On the other hand, a full-scale model for massive grids will certainly be of much interest. This has not been pursued during the elaboration of this first work but it is under development. The arrival of Quantum Optimization to the field of flood risk assessment is very promising but it is still not here. May this small step contribute to the collective work towards this objective.

ACKNOWLEDGMENTS

The author would like to thank Dr. Pedro Pessoa, at the Arizona State University, US, and Prof. Juan Julian Merelo Guervos, at the University of Granada, Spain, for useful discussions and review of this manuscript.

-
- [1] C. Calvo-Sancho, J. Díaz-Fernández, J. J. González-Alemán, A. Halifa-Marín, M. M. Miglietta, C. Azorin-Molina, A. F. Prein, A. Montoro-Mendoza, P. Bolgiani, A. Morata, and M. L. Martín, Human-induced climate change amplification on storm dynamics in Valencia's 2024 catastrophic flash flood, *Nature Communications* **17**, 1492 (2026).
 - [2] O. Conrad, B. Bechtel, M. Bock, H. Dietrich, E. Fischer, L. Gerlitz, J. Wehberg, V. Wichmann, and J. Böhner, System for automated geoscientific analyses (saga) v. 2.1.4, *Geoscientific Model Development* **8**, 1991 (2015).
 - [3] E. F. Combarro and S. González-Castillo, *A Practical Guide to Quantum Machine Learning and Quantum Optimization: Hands-on Approach to Modern Quantum Algorithms* (Packt Publishing, 2023).
 - [4] E. F. Combarro and S. González-Castillo, *A Practical Guide to Quantum Computing: Hands-on approach to quantum computing with Qiskit* (Packt Publishing, 2025).
 - [5] D. O'Malley, An approach to quantum-computational hydrologic inverse analysis, *Scientific Reports* **8**, 6919 (2018).
 - [6] US Army Corps of Engineers, *HEC-RAS, River Analysis System*, Hydrologic Engineering Center (HEC).
 - [7] B. A. Costa and P. Pessoa, Statistical mechanics of unconfined systems: Challenges and lessons, *Physical Sciences Forum* **3**, 10.3390/psf2021003008 (2021).
 - [8] T. G. Project, Geopandas: Geospatial analysis toolkit.
 - [9] D-Wave Systems Inc., Ocean Software SDK.
 - [10] J. Anguiano Olarra, qubodet8 (2026).
 - [11] Qiskit contributors, Qiskit: An open-source framework for quantum computing.